Hyderabad Property Price Prediction

Interview questions you should expect, and how to answer them

Companion to the Data Cleaning and Modelling reports | 27 October 2026

How to use this

The questions are ordered the way an interview usually runs: the story first, then the data, then the modelling, then the awkward ones. The answers are written in the first person and kept short on purpose — an interviewer wants a 20–40 second answer and a chance to follow up, not a monologue.

Three rules that matter more than any individual answer:

- **Lead with the decision, then the evidence.** “I dropped it because the ratio was exactly 1.0 on all 15,472 rows” beats “I thought it might be leakage”.
- **Quote real numbers.** Vague answers read as someone describing a tutorial they followed. The numbers in this document are the ones your notebooks actually produced.
- **Say what did not work and what is still wrong.** Candidates who volunteer the limitations of their own project are rare, and it reads as confidence, not weakness.

The 60-second version (expect this first)

“Tell me about this project.”

“I scraped about 20,700 residential property listings for Hyderabad from MagicBricks into a SQLite database and built a model that predicts the asking price from the listing's features. The work was in four phases: a database audit and join, cleaning, exploration, and modelling.

The cleaning phase was the most interesting. The area column was not in the unit its name claimed, so I reverse-engineered the true unit per row using the site's own published price per square foot. About 37% of listings had coordinates of exactly zero — a null disguised as a valid number — and I found that only because the mean latitude put the dataset in the Indian Ocean. I also found that 86% of the auto-generated descriptions state the price in words, which would have leaked the target into any text feature.

For modelling I compared Linear Regression, Ridge, Random Forest and Gradient Boosting on identical features. The target is log price, because raw price had a skew of 5.82. Gradient Boosting won with a typical error of Rs 53 lakh, about 16% of the asking price, and 66% of predictions within 25%. The trees beat the linear models by a wide margin, which comes down to interactions — a square foot is worth different amounts in different localities, and a linear model cannot express that unless you build the term by hand.”

Then stop talking. Let them pick the thread they care about.

A. Framing the problem

Q1. Why predict asking price rather than the actual sale price?

Because asking price is what the data contains. MagicBricks publishes what sellers ask, not what buyers paid; sale prices are in registration records that are not public at listing level. I am explicit that this is a model of advertised prices, which are systematically optimistic. It is still useful — for flagging a listing as above or below market, or for suggesting a price to a new seller — but it is not a valuation model.

→ *Saying this before they ask shows you know the difference between a dataset and the real world.*

Q2. Is this regression or classification, and how did you decide?

Regression: the target is a continuous rupee amount. I did consider bucketing price into bands, which would make it classification, and rejected it. The buckets would be arbitrary, the model would lose all resolution inside a band, and a listing at the edge of two bands gets no credit for being nearly right. Regression keeps the ordering and the distances.

Q3. Who would use this, and what error would they tolerate?

A seller or an agent setting an asking price, or a portal flagging mispriced listings. For that use the percentage error matters more than the rupee error, which is why I report median absolute percentage error alongside MAE. At 16% typical error it is good enough to sanity-check a price, not to sign a cheque on.

Q4. Why was the data scraped in price bands, and what does that cost you?

The site paginates and caps how deep you can go, so scraping band by band (1–50 lakh, 50–80 lakh, 80 lakh–1 crore, upward) was the only way to get coverage across the price range. The cost is that the mix of cheap and expensive listings in my data reflects how many pages each band offered, not the real market. That is fine for a conditional model — given these features, what price — but it means I cannot make claims about the Hyderabad market overall, and it is why my error is worst at the extremes, where coverage is thinnest.

→ *This is a sampling-bias question in disguise. Name it as sampling bias.*

B. Data collection, SQL and the join

Q5. Walk me through how the two tables were combined.

Two tables: *listings*, one row per property with 57 columns, and *locality_ratings*, one row per locality with 38 columns of amenity ratings and published price statistics. I joined them with a LEFT JOIN from listings on locality_id.

Q6. Why a left join and not an inner join?

An inner join would silently delete every listing whose locality had no ratings record. Those listings still have a price, a size and a type — they are perfectly good training examples, they just lack the locality extras. A left join keeps them and leaves the locality columns null, which I then handle as missing data with an indicator column. Roughly 7% of rows are in that situation, so an inner join would have thrown away about 1,400 listings for no reason.

→ *Classic question. The answer is always 'because the right table is optional information, not a filter'.*

Q7. How did you check the join did not duplicate rows?

By comparing the row count before and after: 20,700 before, 20,700 after. If locality_id had not been unique in the ratings table, a left join would have multiplied rows, and the count would have risen. I also confirmed locality_ratings had one row per locality.

Q8. Why extract to CSV instead of querying SQLite directly?

For this project size it was convenience — pandas and the whole scikit-learn stack work directly on a DataFrame, and the extract is a single reproducible query. If the data were larger, or refreshed on a schedule, I would push the join and the filtering into SQL and read only the columns I need, because moving 260 MB of CSV around is wasteful.

C. Cleaning (the part with the best stories)

Q9. What was the hardest data quality problem you found?

The area column. It was named covered_area_sqft but it held whatever unit the seller selected — square feet, square yards, square metres, even cents — with a companion column naming the unit, sometimes as an unreadable numeric code. About 10% of rows had the area understated roughly ninefold. Since area is the strongest predictor of price, that was the most damaging error in the dataset.

Q10. How did you fix it without an external reference?

The site publishes its own price per square foot. So for each row, (price / area) divided by that published figure gives the exact conversion factor the site itself applied. Grouped by the unit column, the median of that ratio identified every code: 1.0, 9.0, 10.764, 435.6. Then the same ratio per row exposed a bigger problem — on 463 rows the unit label itself was wrong, square yards labelled as feet and vice versa. I trusted the arithmetic over the metadata and used the label only as a fallback.

→ *The transferable line: 'when two sources disagree, find a third that is derived differently'.*

Q11. You said 37% of coordinates were zero. Why is that worse than missing?

Because (0, 0) is numerically valid, so nothing complains. It silently drags every statistic computed over it — my mean latitude was 10.9, which is in the Indian Ocean, and that is how I spotted it. A null at least announces itself. I tested coordinates against a bounding box for Hyderabad's real extent, recovered four rows where latitude and longitude were simply transposed, nulled the genuine geocoding failures, and let the rest fall back to their locality's centre point — with a flag recording the substitution so the model can tell an exact coordinate from an approximate one.

Q12. How did you handle duplicates?

The obvious check found nothing, which was itself the finding: columns like the detail URL and the scrape timestamp are unique per scrape by construction, so every row looks distinct. I re-ran it on a content key — locality, type, bedrooms, area, price, floor, project — and found 517 rows in 236 groups. I sorted by listing date descending before dropping, so the surviving copy carries the most recent asking price.

Q13. Why did you not just drop the 1% and 99% tails of price to remove outliers?

I checked, and the percentile was not trustworthy. The 99th percentile of implied price per square foot was Rs 75,969, when Hyderabad's most expensive real estate trades near Rs 25,000–30,000. The tail was already contaminated by seller typing errors, so the percentile computed over it inherits the corruption. I used domain bounds instead — Rs 1,500 to 40,000 per square foot, and 200 to 20,000 square feet — and filtered on price per square foot rather than on price, because Rs 40 crore is ordinary at 18,000 square feet and absurd at 1,200.

→ *A strong answer. It shows you know a statistic computed over bad data inherits the badness.*

Q14. Did you drop every column that was mostly empty?

Deliberately not. Parking was 75% missing, power status 73%, water status 72%, and I kept them through cleaning because a blank parking field most likely means there is no parking — the missingness is itself a signal, and a blanket 'drop over 50% null' rule would have destroyed it. I did set them aside for the first model, because they need string parsing, but that is a scope decision I can reverse, not a deletion.

D. EDA and the target

Q15. Why log-transform the price?

Because raw price had a skew of 5.82, and logging it brought that to 0.09. The median listing is about Rs 1.5 crore but the maximum is Rs 49 crore, so a thin tail of expensive property stretches far right. Linear models minimise squared error, so those few listings dominate the loss and the model ends up fitting mansions instead of the flats that are most of the data. Logging compresses the large values, and all four models improved.

Q16. If you train on log price, how do you report the error?

I convert the predictions back with expm1 before scoring, so MAE and RMSE are in rupees and mean something to a human — Rs 53 lakh typically. R-squared I report on the log scale, because that is the scale the model was actually fitted on. Reporting a rupee R-squared from a log-fitted model mixes the two and is easy to get wrong.

→ A frequent follow-up: 'does expm1 of the mean equal the mean?' No — undoing a log gives the geometric mean, so predictions are slightly conservative. Mentioning it unprompted lands well.

Q17. You ran EDA after splitting. Why that order?

Because EDA is not passive — the imputation values, the category groupings and the drop list all came out of what I saw. If I had looked at all 19,340 rows first, those decisions would carry information from the test rows, and the final score would flatter the model. So I split first with a fixed seed and every number I looked at came from the 15,472 training rows only.

→ This single answer separates people who understand leakage from people who have heard of it.

Q18. Why did you not stratify the split on price?

Because the price distribution is an artefact of how I scraped — band by band. Stratifying on it would lock that artefact into both halves of the split and make the test set agree with the training set by construction. A plain random split with a fixed seed is the honest choice. I would stratify if a class were genuinely rare and I needed it represented, which is more of a classification concern.

Q19. What did the correlations tell you?

Covered area was the strongest honest predictor at 0.67 against log price, then bathrooms 0.54, bedrooms 0.51, and the locality's published average price around 0.31–0.34. It also told me bedroom count alone is weak — the boxplots overlap heavily, because a 3-BHK in Kondapur and a 3-BHK on the outskirts are the same room count at very different prices. That is what pushed me to get the locality representation right.

E. Feature engineering

Q20. Why so little feature engineering?

Because the baseline did not need it and every transformation is a place for a bug to hide. I made exactly two repairs: cleaning the facing column and collapsing possession from 48 levels to three. No interactions, no clustering, no text features. The log transform and an honest validation protocol bought far more than invented features would have, and now I have a trustworthy baseline to measure future features against.

→ Interviewers respect restraint when you can justify it. Without the justification it reads as laziness.

Q21. Walk me through your pipeline.

A ColumnTransformer with two branches. Numeric: median imputation with add_indicator, then StandardScaler. Categorical: constant fill with 'Unknown', then OneHotEncoder with handle_unknown='ignore' and min_frequency=10. That sits in a Pipeline with the model, so 23 features become 67 columns, and cross-validation refits every imputation value, category list and scaling parameter on each fold.

Q22. Why does the pipeline matter? You could impute with pandas first.

Because a median computed with pandas over the whole training set leaks across folds. When cross-validation holds out a fifth of the rows, it scores them using a median that was partly computed from them. Inside a pipeline the median is recomputed from the four-fifths only. The pipeline also guarantees the same preprocessing at prediction time, so I cannot forget a step on new data — the saved model file carries its own preprocessing.

Q23. Why median imputation rather than mean, and why the indicator columns?

Median because these columns are skewed, and the mean of a skewed column is dragged by the tail. The indicator because the missingness is informative, and I proved it: the floor number is missing for 99.9% of villas and 50.6% of houses, against 4.6% of apartments. 'Which floor?' is a meaningless question for a standalone house, so the blank is really telling me the building type. Filling it and marking it lets the model use both facts.

→ *The 99.9% figure is the most memorable number in your project. Use it.*

Q24. Why scale the features when two of your four models are trees?

Trees do not need it and are unaffected by it, because a split is a threshold on one feature and rescaling moves the threshold identically. Ridge does need it: it penalises the squared size of the coefficients, so without scaling the penalty would fall hardest on whichever feature happens to use small units — latitude would be punished more than area simply because of its range. Using one shared preprocessor for all four models costs nothing and is what makes the comparison fair.

Q25. How do you handle a category in the test set that never appeared in training?

handle_unknown='ignore' encodes it as all zeros rather than raising. I also set min_frequency=10, which pools categories with under 10 training rows into one 'infrequent' column — so rare categories get a shared column instead of a column each, and a genuinely new category lands in familiar territory rather than nowhere.

F. Leakage (expect several of these)

Q26. What is data leakage, and where did you find it?

Leakage is any feature that carries information you would not have at prediction time. I found it in four forms. One, price_per_sqft, the site's own figure, which is the target divided by area. Two, price_per_sqft_true, the same quantity recomputed during cleaning under a new name — I verified it by dividing price-by-area by it and getting exactly 1.0 on all 15,472 rows. Three, price_label, a text version of the price. Four, and the one I nearly missed: the auto-generated description text states the price in words in 86% of listings — 'this apartment is sold at price of Rs 46.9 Lac'.

→ *Four distinct forms, with a proof for one of them. This is your strongest single answer.*

Q27. How would you have caught the leakage if you had not been looking for it?

The symptom is a score that is too good. An R-squared above 0.95 on asking prices from 23 listing fields is not plausible, and a feature importance chart topped by something I cannot explain in words is the other tell. My importance chart is led by covered area and the locality price aggregates, which is exactly what a property valuer would say matters — a boring ranking is good news.

Q28. You kept the locality average price. Is that not leakage too?

I thought about this carefully and concluded no, with a caveat I document. The test for leakage is whether the value would be available at prediction time. The locality averages are published by the website before any individual listing exists, so they would be. They are also not target encoding — I am not computing the mean price of my own training rows per locality, which would leak and would have to be fitted inside each fold. The caveat is that the site computed those averages over a pool of listings that overlaps mine, so my validation is mildly optimistic, most of all in localities with few listings behind the average. I documented that rather than using it as an excuse to throw away a genuinely useful feature — over-pruning out of caution costs accuracy just as surely.

→ *The caveat is what makes this answer credible. Do not leave it out.*

Q29. If you did add target encoding for locality, how would you do it safely?

Inside the cross-validation loop, never before it. The encoder has to be a transformer fitted on the training folds only, so the mean price for a locality is computed without the rows being scored. I would also smooth it towards the global mean for localities with few listings, otherwise a locality with two expensive listings gets an extreme and unreliable value. That is the main improvement I have queued, and it is exactly the kind of feature that looks brilliant and is worthless if fitted outside the folds.

G. Models and the theory behind them

Q30. Why those four models?

They ladder up in capability, which makes the comparison itself informative. Linear Regression is the honest floor with no tuning. Ridge is the same model with a penalty, so the gap between them tells me whether collinearity is a problem. Random Forest and Gradient Boosting are the two standard tree ensembles, and they work in opposite ways — one averages many deep independent trees to cut variance, the other adds shallow trees sequentially, each fitting the previous one's errors, to cut bias.

Q31. Why did the tree models win, specifically?

Interactions. A linear model has to assume an extra square foot adds the same number of rupees everywhere in Hyderabad, and it plainly does not — a square foot in Jubilee Hills and one on the outskirts are different goods. That is an interaction between area and locality, and a linear model cannot represent it unless I build the combined term by hand. A tree splits on the locality price level first and then on area inside each branch, so it discovers the structure itself. On the test set the gap is R-squared 0.615 for Linear against 0.776 for Gradient Boosting.

→ *This is the question that most separates a copied project from an understood one.*

Q32. Explain the difference between bagging and boosting.

Bagging — Random Forest — trains many deep trees in parallel, each on a random sample of rows and a random subset of features, then averages them. Each tree overfits, but their errors are largely independent, so averaging cancels them: it attacks variance. Boosting trains shallow trees in sequence, each one fitted on what the previous ones got wrong, and adds them up with a small learning rate: it attacks bias. The practical consequences are that a forest is hard to overfit and easy to parallelise, while boosting is usually more accurate but sensitive to its learning rate and tree count, which trade off against each other.

Q33. Ridge barely improved on Linear Regression. Why, and is that a problem?

It is a good sign, not a problem. Ridge penalises large coefficients, which mostly helps when features are collinear or there are many of them relative to the rows. I had already dropped 16 locality rating columns that all move together, so there was little collinearity left to fix, and I have 67 columns against 15,472 rows. If Ridge had rescued a badly behaved Linear Regression, that would have pointed to a problem in my feature set rather than a virtue in Ridge.

Q34. What is the difference between Ridge and Lasso, and why Ridge?

Both add a penalty on coefficient size. Ridge penalises the sum of squares, which shrinks coefficients towards zero but never to zero. Lasso penalises the sum of absolute values, which drives some coefficients exactly to zero, so it also selects features. I used Ridge because I had already selected features deliberately and with reasons — I did not want an algorithm silently deleting a column I had argued for. Lasso would be the natural choice if I had hundreds of features and no view on which mattered.

Q35. How did you tune the hyper-parameters?

GridSearchCV on the training set with the same 5-fold split, scored on negative MAE of the log target — I optimise the metric I report. Small grids, each parameter chosen because it controls something specific: alpha for Ridge across four orders of magnitude; max_depth and min_samples_leaf for the forest, which control overfitting, since a leaf holding one row has memorised it; and n_estimators, learning_rate and max_depth for boosting, which trade off against each other.

Q36. What would you reach for next, and why not now?

HistGradientBoostingRegressor or XGBoost. Both handle missing values natively, which would remove most of my imputation code, and both train far faster than plain Gradient Boosting. I did not start there because the point of this phase was a baseline I fully understand. Reaching for the strongest model before you have a trustworthy protocol means you cannot tell whether an improvement came from the model or from a mistake.

Q37. Why not a neural network?

On 15,000 rows of mostly tabular, mostly categorical data, gradient-boosted trees are the better tool and are known to be — they handle mixed types, need little preprocessing and do not need this much data. A neural network would need more rows, more tuning and more regularisation to reach the same place, and it would be harder to explain to whoever uses the number.

H. Validation and metrics

Q38. Why cross-validation instead of a single validation split?

With one split, the score depends on which rows happened to land in it, and I was making several decisions in sequence — feature sets, locality encoding, hyper-parameters. Each decision against a single split risks fitting that split. Five-fold gives every training row a prediction from a model that never saw it, so the comparison is more stable and I am not tuning against one lucky sample.

Q39. Why report MAE, RMSE, R-squared and MedAPE instead of one number?

They answer different questions. MAE is the typical miss and the one a human can judge: Rs 53 lakh. RMSE squares the errors first, so it punishes large misses harder — RMSE far above MAE tells me a few predictions are badly wrong, which is exactly what happens at the expensive end. R-squared is the share of variation explained, 0.78 on the log scale. And MedAPE is the miss as a percentage, 16%, which the rupee metrics cannot see: Rs 20 lakh is a disaster on a Rs 50 lakh flat and a rounding error on a Rs 10 crore villa.

Q40. Why the median absolute percentage error rather than the mean?

Because a percentage error has the price in its denominator, so a handful of very cheap listings can produce enormous percentages and dominate a mean. The median is robust to that. The same reasoning applies to the mean absolute percentage error's well known failure on near-zero targets.

Q41. What was your baseline, and why does a baseline matter?

Always predict the median price. It scores about Rs 108 lakh MAE on the test set, against Rs 53 lakh for Gradient Boosting. Without that number, a good-looking R-squared means nothing — you cannot tell whether the model learned the data or just learned roughly where the middle of the price range sits. A model that cannot beat the median by a wide margin has learned nothing.

→ *Having a baseline at all puts you ahead of most beginner projects.*

Q42. How many times did you use the test set?

Once, at the very end, after every choice had been made — features, encoding, hyper-parameters, all decided by cross-validation on the training set. The test numbers came out close to the cross-validation numbers, which is itself the result worth reporting: it means nothing was accidentally tuned against the test set.

Q43. What would you do if the test score had been much worse than cross-validation?

I would not tune until it improved — that converts the test set into a validation set and I would have nothing honest left. I would look for a cause: leakage inflating the cross-validation score, a group effect where near-duplicate listings straddle the split, or a shift between the two halves. The duplicate work in cleaning was partly for this reason — identical feature vectors with identical prices split across train and test inflate the test score regardless of why they are duplicated.

I. Results and error analysis

Q44. How good is your model, honestly?

Rs 53 lakh typical error, 16% of the asking price, R-squared 0.78 on log price, with 66% of predictions within 25% and 34% within 10%. That is a reasonable baseline and not a production valuation tool. The ceiling here is set by the data: two identical flats on the same street can legitimately differ by 30% on floor, view, finish and how motivated the seller is, and none of that is in the 23 columns I have.

Q45. Where does the model fail worst?

At both ends of the market, in percentage terms — the cheapest and the most expensive listings. Two reasons, and they are the same reason really: the price-band scrape covers those segments thinly, and expensive property is the most individual, so a Rs 20 crore villa's price depends on finishes, plot shape and view that no column captures. I broke the error down by price band rather than quoting one average, because the average hides this.

Q46. What does your feature importance chart show, and do you trust it?

Covered area dominates, then the locality price aggregates, then bathrooms and bedrooms. I trust the ordering as a sanity check — it is what a valuer would say matters, and that is evidence no leaked column is doing the work. I do not over-read the exact numbers: tree importances are biased towards high-cardinality and continuous features, and with one-hot encoding the importance of a category gets split across its columns. For a real claim about a single feature I would use permutation importance on held-out data instead.

→ *Volunteering the known bias in tree importances is a strong signal.*

Q47. Does your model predict a 2-BHK in a locality it has never seen?

Yes, and that was a deliberate design choice. If I had one-hot encoded the 2,177 localities, an unseen locality would be all zeros and the model would fall back to the global average. Because I used the locality's published statistics instead, a new locality still arrives with a price level, a rank and coordinates, and the prediction stays sensible. That is the practical argument for the numeric encoding, separate from the accuracy argument.

J. The awkward questions

Q48. You have 2,177 localities and you one-hot encoded none of them. Is that not throwing away the most important feature?

It would be, if I had thrown the information away — but I replaced it with four numeric columns that summarise the same thing: the locality's published price per square foot, its rank, its quarterly growth and its overall rating. And I measured it rather than assuming: I ran all three options with the one-hot encoder inside the pipeline so the category list was rebuilt per fold. The published numbers won, adding one-hot on top changed almost nothing, and one-hot alone was clearly worse. One well-chosen number beat a hundred indicator columns.

→ *Have the three-row experiment table ready. This question is a trap only if you have no evidence.*

Q49. Your R-squared is not very high. Why should I be impressed?

Because the number is honest, and I can tell you exactly what limits it. I removed four forms of leakage, any one of which would have pushed R-squared past 0.95 and produced a model that cannot predict anything. I can also tell you what the remaining error is made of — thin coverage at the market's extremes, and genuine price variation that 23 listing columns cannot see. A lower number I can account for is worth more than a higher one I cannot.

Q50. What is the single biggest weakness of this project?

That it predicts asking prices and cannot be validated against what anything actually sold for. Everything else — the sparse amenity columns, the missing text features, the locality-resolution coordinates — is fixable with more work. That one is a limit of the data source, and the honest thing is to state it rather than let someone assume the model knows market value.

Q51. What would you do differently if you started again?

Three things. I would write the cleaning as re-runnable functions from the start — I hit three bugs that all came from notebook state depending on execution order, including an inplace drop that broke on re-run and a drop missing its reassignment that appeared to work while changing nothing. I would scrape a held-out time period rather than splitting randomly, so I could test on genuinely later listings. And I would capture the locality statistics with a timestamp, so I could prove they predate the listings instead of arguing it.

Q52. How would you put this into production?

The model is saved with joblib and carries its own preprocessing, so serving it is one predict call behind a small API — no risk of forgetting a transformation. The real work is around it: validating the input ranges, because the model is only trustworthy between 200 and 20,000 square feet and Rs 1,500–40,000 per square foot; returning a range rather than a single number, given 16% typical error; refreshing the locality statistics, which go stale; and monitoring the input distribution, because a model trained on a price-band scrape will drift as the market moves.

K. Five questions to ask them

Having questions ready reads as interest. These also happen to be the ones a data team will enjoy answering.

- How do you decide a model is good enough to ship here — is there a metric threshold, or is it judgement?
- When you find leakage in production, how do you usually find out — monitoring, or someone noticing the numbers are too good?
- How much of the team's time goes on data quality versus modelling? Mine was about 70% cleaning.
- Do you retrain on a schedule or on a trigger, and what triggers it?
- For tabular problems like this, where has the team landed between gradient boosting and deep learning?

One-page cheat sheet

Question	The number
Rows, before and after cleaning	20,700 → 19,340 (95% retained)
Columns, before and after	94 → 71, then 23 features used
Train / test	15,472 / 3,868, seed 42
Price skew, raw → logged	5.82 → 0.09
Strongest honest feature	covered area, r = 0.67 on log price
Localities	2,177; only 129 with 20+ listings
Leakage removed	4 forms, incl. price stated in 86% of descriptions
Floor missing, villas vs apartments	99.9% vs 4.6%
Best model	Gradient Boosting
Typical error	Rs 53 lakh (16% of price)
R-squared, log price	0.78
Within 25% / 10% of true price	66% / 34%
Beats median baseline by	Rs 54 lakh of MAE

If you remember only four things: the area unit was reverse-engineered from the site's own price per square foot; 86% of descriptions state the price, so text was leakage; four locality numbers beat 2,177 one-hot columns, and it was measured; and the floor number is missing for 99.9% of villas, which makes the blank a feature.